Adversarial State-Space Optimization in Chess Environments: A Transparent, Verified and Benchmarked Classical Search Engine

Baviri Setty Sai Deevan
Dept. of CSE (Cyber Security)
Rajalakshmi Engineering College
Chennai, India

Monish Priyan M
Dept. of CSE (Cyber Security)
Rajalakshmi Engineering College
Chennai, India

S. Bhuvana
Dept. of CSE (Cyber Security)
Rajalakshmi Engineering College
Chennai, India

***Abstract*—Modern chess engines derive much of their strength from neural networks whose decisions cannot be inspected, which makes them poor vehicles for teaching or auditing adversarial search. This paper presents a classical chess engine, written from first principles and run entirely in the browser, that models chess as a fully observable, deterministic, zero-sum state space and makes the cost of every search technique measurable. The engine combines Negamax with alpha-beta pruning, iterative deepening, quiescence search, MVV-LVA move ordering, null-move pruning, contempt and a fixed-size Zobrist-hashed transposition table. Its move generator matches the published perft counts on seven standard positions, and 53 automated tests gate every deployment. Across 917 fixed-depth searches on 17 positions, alpha-beta, ordering and the transposition table returned exactly the minimax move and score in all 79 comparable cases while shrinking the depth-4 tree by a geometric mean factor of 110.6, and the effective branching factor fell from 24–27 to 2–5. Null-move pruning was worth 132 Elo points in self-play, and the engine rates about 1600–1700 on Stockfish 19's calibrated scale at one second per move. The deployed application is threat-modelled with STRIDE and its controls are evaluated.**

***Index Terms*—adversarial search, alpha-beta pruning, chess, game-tree search, transposition table, null-move pruning, explainable AI, threat modeling**

I. Introduction

Game playing has served as a proving ground for artificial intelligence since Shannon described how a computer could play chess by searching a tree of future positions and scoring its leaves with an evaluation function [1]. Deep Blue's 1997 victory over the world champion rested on massively parallel alpha-beta search and a hand-tuned evaluation [2]. The strongest engines have since moved towards learned evaluation: AlphaZero learned chess from self-play with a deep network guiding Monte-Carlo tree search [3], and Stockfish pairs alpha-beta search with an efficiently updatable neural network (NNUE) [4], [5].

Learned evaluation is powerful but opaque. An engine reports a score and a principal variation, but not what pruning, ordering or caching contributed to them, and the costs of these classical techniques are rarely measured together on the same positions with the same code. Course-level engines often rely on an external move-generation library that their authors neither implement nor verify, and AI artefacts published as web applications are seldom analysed as attack surfaces.

This paper addresses these gaps with an engine whose every component is written from first principles, verified against published reference data and measured in isolation. Its contributions are: (i) a browser-based engine with a *teaching mode* that searches any position under independently switchable techniques; (ii) a verification regime of perft on seven standard positions, 53 automated tests and an equivalence check against plain minimax; (iii) a measurement of eight configurations over 17 positions and depths 1–7 (917 searches); (iv) a game-based evaluation with self-play matches and a 128-game benchmark against calibrated Stockfish; (v) two defects found by measurement rather than inspection; and (vi) a STRIDE threat model of the deployed application.

II. Related Work

Shannon [1] framed chess as minimax search over a depth-limited tree. Knuth and Moore [6] proved that alpha-beta returns the exact minimax value and that, under perfect move ordering, it examines about $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves instead of b^d for branching factor b and depth d; Marsland [7] reviewed the pruning methods that followed. Korf [8] showed that iterative deepening is asymptotically optimal in time with linear memory. Zobrist [9] introduced XOR-based position hashing, the basis of transposition tables, and Schaeffer [10] found move ordering to dominate search efficiency. Beal [11] generalised quiescence search against the horizon effect, and Donninger [12] described null-move pruning, which is unsound in zugzwang. Deep Blue [2], Monte-Carlo tree search [13], AlphaZero [3] and NNUE-based Stockfish [4], [5] optimise strength; this work instead optimises transparency and measurability.

III. Problem Formulation

Following Russell and Norvig [14], chess is a game defined by an initial state, the player to move, the legal actions $A(s)$, a transition model $R(s, a)$, a terminal test and a utility. The environment is fully observable, deterministic, sequential,

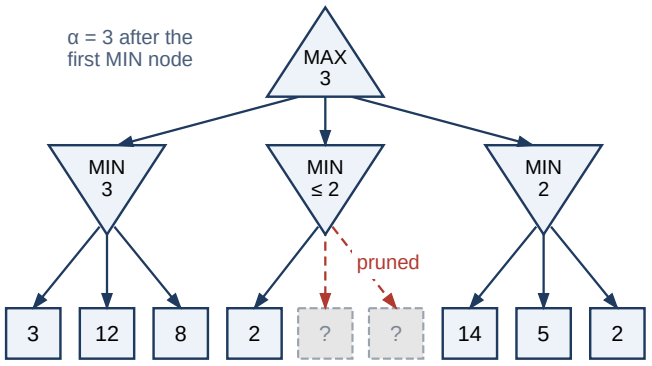


Fig. 1. Minimax tree with alpha-beta pruning. Once the first MIN node returns 3, the second cannot exceed 2 after its first leaf, so its remaining leaves need not be searched.

discrete and competitive. The depth-limited Negamax value of a state from the side to move is

$$v_d(s) = \begin{cases} U(s) & \text{if } s \text{ is terminal,} \\ E(s) & \text{if } d = 0, \\ \max\limits_{a \in A(s)} -v_{d-1}\big(R(s,a)\big) & \text{otherwise,} \end{cases} \tag{1}$$

where U is the terminal utility and E a static evaluation, both from the mover's perspective. Given a position and a time budget T, the agent must return the move maximising v_d for the largest depth d it can complete within T, using a self-built, verifiably correct move generator, and must make the cost of each optimisation measurable while preserving v_d exactly wherever the technique is sound. Fig. 1 illustrates the pruning that makes this feasible.

IV. System Design

A. Architecture

The system runs entirely on the client (Fig. 2). A React interface on the browser's main thread holds the game record, clocks and position history; the engine runs in a dedicated Web Worker that receives a serialised board snapshot and streams progress back, so the interface never blocks. A GitHub Actions pipeline installs dependencies from a lockfile, runs the test suite, builds the bundle and deploys it to Firebase Hosting. Fig. 3 decomposes the system into six processes: move generation (1.0), evaluation (2.0), adversarial search (3.0), search optimisation (4.0), the interface (5.0) and verification (6.0), with the board state (D1), evaluation tables (D3) and transposition table (D4) as data stores.

B. Board, Move Generation and Evaluation

The board is a 64-element array with side to move, castling rights, en passant square, move clocks and a history stack for exact make/unmake. Pseudo-legal moves are generated per piece type and filtered by testing whether the mover's king is attacked. Positions are read from Forsyth–Edwards Notation (FEN), validated before use, and moves are written in Standard Algebraic Notation (SAN). The evaluation returns centipawns

Algorithm 1 Negamax with Alpha-Beta, Null Move and TT

function NEGAMAX($d, ply, \alpha, \beta, allowNull$)
  if repetition **or** $halfMoveClock \geq 100$ **then return** DRAW
  $key \leftarrow$ ZOBRIST(s); $(v, m_{tt}) \leftarrow$ TT.LOOKUP($key, d, \alpha, \beta, ply$)
  if $v \neq$ NULL **then return** v
  if $d \leq 0$ **then return** QUIESCENCE(α, β)
  $M \leftarrow$ LEGALMOVES(s)
  if $M = \emptyset$ **then return if** in check **then** $-20000 + ply$ **else** DRAW
  if $allowNull \wedge \neg$check $\wedge\, d \geq 4 \wedge$ HASPIECES **then**
    $v \leftarrow -$NULLSEARCH($d - 1 - R, ply + 1, \beta$)
    if $v \geq \beta$ **then return** β
  $\alpha_0 \leftarrow \alpha$; $best \leftarrow -\infty$
  for $m \in$ ORDER(M, m_{tt}) **do**
    MAKE(m); $v \leftarrow -$NEGAMAX($d - 1, ply + 1, -\beta, -\alpha,$ TRUE); UNMAKE(m)
    if $v > best$ **then** $best \leftarrow v$; $m^* \leftarrow m$
    $\alpha \leftarrow \max(\alpha, v)$
    if $\alpha \geq \beta$ **then break**
  TT.STORE($best$, bound from α_0 and β, m^*)
  return $best$

from White's perspective and sums five terms: material (P 100, N 320, B 330, R 500, Q 900), piece-square tables mirrored for Black, pawn structure (-15 per doubled and -20 per isolated pawn), king safety (-30 on an open file, $+10$ per shield pawn when castled) and mobility ($+2$ per knight or bishop move); the material values and tables follow the Simplified Evaluation Function [15]. The same code produces the score used by the search and the per-term breakdown shown to the user.

C. Search

Algorithm 1 gives the core search. An iterative-deepening driver calls it for depths $1, 2, \ldots, 12$ until the budget expires (2,000 ms without a clock), keeping the move of the last completed iteration. Repetitions and the fifty-move rule are detected before the transposition-table probe, and draws are scored with a contempt $c \in [-100, 100]$ centipawns. Moves are ordered hash move first, then captures by Most Valuable Victim–Least Valuable Attacker (MVV-LVA), then promotions; quiescence searches captures only, with a stand-pat bound. Null-move pruning searches the opponent's reply with a null window at β, $R = 2$ plies shallower, and is tried only with at least four plies remaining, never in check, never twice in a row and not with king and pawns only. The four-ply guard was chosen by experiment (Section VI-D).

D. Fixed-Size Transposition Table and Teaching Search

The table holds 2^{19} slots in seven parallel typed arrays at 19 bytes per slot, about 10 MB, allocated once and never grown. A 64-bit Zobrist key [9] is kept as two 32-bit halves; a slot is selected by the low key bits and verified against all 64. On a collision the same position is refreshed, an entry from an earlier search yields, and within one search the deeper result stays. A separate fixed-depth teaching search lets alpha-beta, ordering, the table, quiescence and null move be switched independently (Fig. 4). It is deliberately repetition-blind, so unpruned minimax visits exactly $\sum_{i=0}^{d} \mathrm{perft}(i)$ nodes, turning a performance figure into a correctness check.

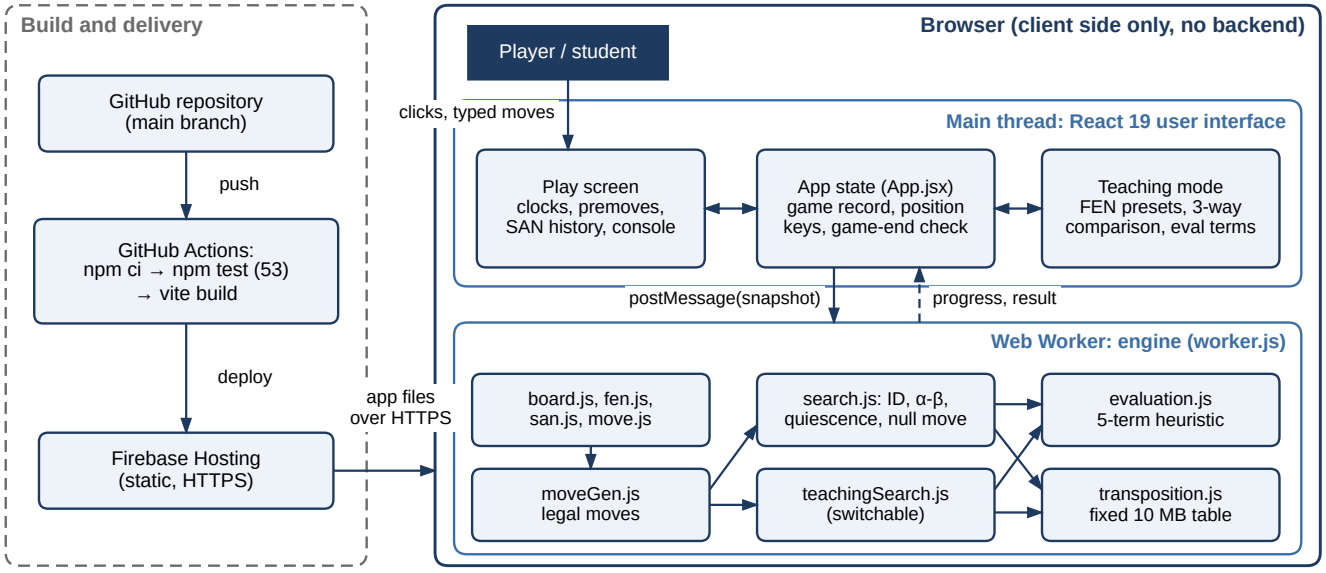


Fig. 2. System architecture. All computation is client-side; the engine runs in a Web Worker, and the build pipeline deploys only after the test suite passes.

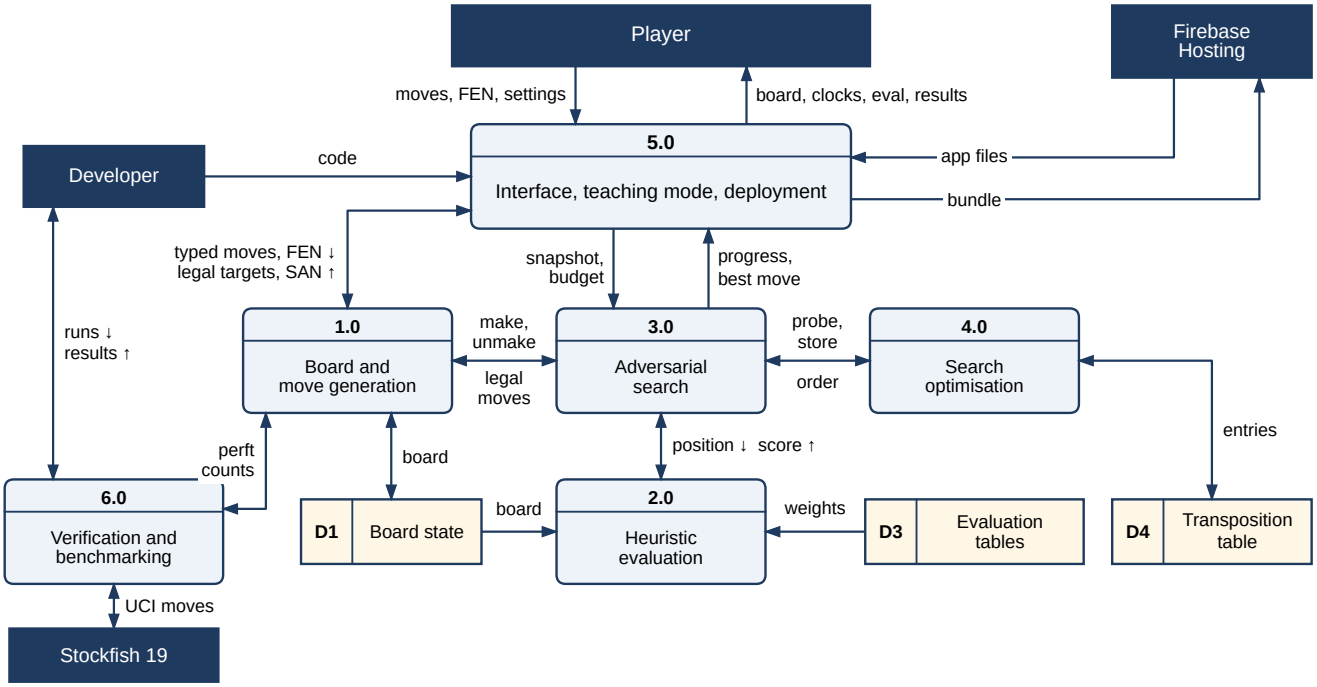


Fig. 3. Level-1 data flow diagram. Dark rectangles are external entities, rounded boxes are numbered processes and light rectangles are data stores.

V. Verification

Perft counts the leaves of the legal-move tree to a fixed depth, so any error in move generation changes the count. The generator matches every published value [16] in Table I—30 depths over seven positions and 17.0 million leaves—covering castling, en passant including a discovered-check case, every promotion piece and pins. A suite of 53 tests runs in about four seconds and precedes every build, so a failing test blocks deployment; beyond perft it covers FEN and SAN, repetition keys, colour symmetry of every evaluation term, the transposition table's rules, short mates with and without null move, contempt and the teaching search's perft identity.

VI. Experimental Evaluation

A. Setup

The teaching search was run on 17 positions—four openings, four busy middlegames, five tactical positions including Win at Chess (WAC) #1 and #4 [17], and four endgames

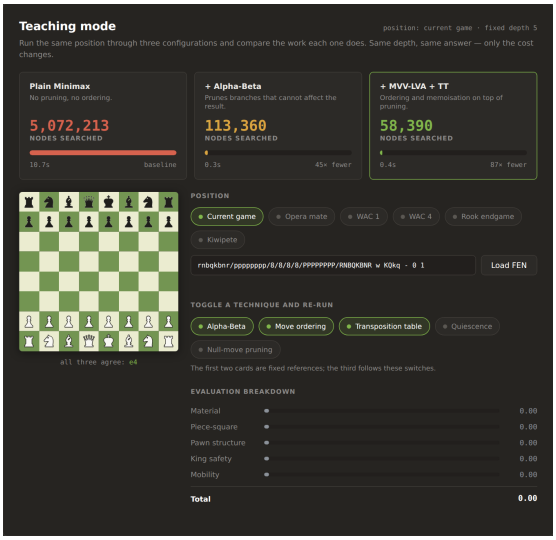


Fig. 4. Teaching mode on the starting position at depth 5: plain minimax, alpha-beta, and alpha-beta with ordering and the transposition table, with FEN presets and technique toggles.

TABLE I
PERFT COUNTS MATCHED ON THE STANDARD POSITIONS

Position	$d=1$	$d=2$	$d=3$	$d=4$	$d=5$
Start	20	400	8,902	197,281	4,865,609
Kiwipete	48	2,039	97,862	4,085,603	–
Position 3	14	191	2,812	43,238	674,624
Position 4	6	264	9,467	422,333	–
Position 4, mirrored	6	264	9,467	422,333	–
Position 5	44	1,486	62,379	2,103,487	–
Position 6	46	2,079	89,890	3,894,594	–

including Fine #70 [18]—at depths 1 to 7 under eight configurations: plain minimax (C0); alpha-beta (C1); C1 with MVV-LVA ordering (C2) or the transposition table (C3); C1 with both (C4); and C4 with null move (C5), quiescence (C6) or both (C7). A configuration stopped deepening on a position once a search exceeded 30 s; 51 of the 917 searches hit that limit. Node counts are exact; times were measured on one core.

B. Node Reduction and Branching Factor

At depth 5 on the starting position (Fig. 5), C0 visits 5,072,213 nodes, exactly $1+20+400+8{,}902+197{,}281+4{,}865{,}609$; C1 visits 113,360 (45× fewer) and C4 58,390 (87× fewer). All three choose 1.e4, and the counts were reproduced exactly in the deployed interface (Fig. 4).

Table II reports the reduction relative to C0 as a geometric mean over the positions C0 finished, and Fig. 6 plots depths 1–4, where all 17 positions are included. Alpha-beta alone saves about 20× at depth 4; ordering multiplies that roughly five-fold, because alpha-beta can cut a branch only after seeing a good enough move. On Kiwipete at depth 5, C1 needs 6.6 million nodes and C2 75 thousand. The transposition table adds more with every ply as transpositions multiply.

Table III gives the effective branching factor, the ratio of nodes between successive depths. C0 pays the full branching

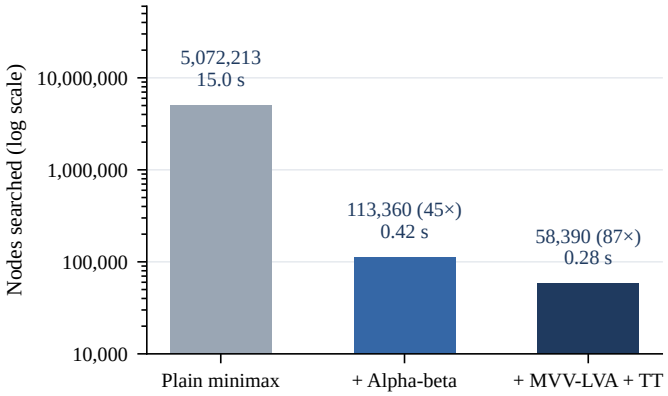


Fig. 5. Nodes searched on the starting position at depth 5 by plain minimax, alpha-beta, and alpha-beta with ordering and the transposition table.

TABLE II
NODE REDUCTION RELATIVE TO C0 (GEOMETRIC MEAN)

Config.	$d=2$	$d=3$	$d=4$	$d=5$	$d=6$	$d=7$
C1	2.6	5.8	20.8	17.5	20.6	28.5
C2	6.2	17.3	102.6	48.4	51.4	31.0
C3	2.6	5.8	23.2	26.3	47.5	169.5
C4	6.2	17.3	110.6	70.1	105.7	187.2
C5	6.2	17.3	110.6	105.1	116.3	187.2
Positions	17	17	17	6	3	2

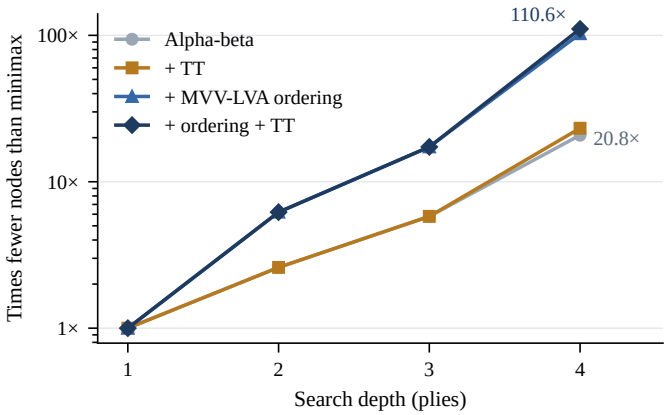


Fig. 6. Node reduction relative to plain minimax by depth (geometric mean over all 17 positions).

factor of chess, 24–27; with ordering, the table and null move a ply costs only 2–5 times more from depth 4, near $\sqrt{b}$, as alpha-beta with perfect ordering predicts. The alternation in the C1 row is the odd–even effect of the $b^{\lceil d/2\rceil}+b^{\lfloor d/2\rfloor}$ bound [6].

C. Equivalence and Speed per Node

For all 79 position–depth pairs C0 finished, C1–C4 returned exactly C0's score and move (79/79), and with the four-ply guard C5 also agreed in all 79. Quiescence answers a different question—it searches captures beyond the nominal depth—and C6 matched C0's move in 52 of the 79 cases, as intended. Fewer nodes does not imply less time: ordering costs over

TABLE III
EFFECTIVE BRANCHING FACTOR

Config.	1→2	2→3	3→4	4→5	5→6	6→7
C0	23.64	27.33	24.16	14.30*	8.14*	6.59*
C1	9.26	11.99	6.75	10.45	4.04	6.87
C4	3.83	9.72	3.79	6.92	2.16	3.86
C5	3.83	9.72	3.79	5.14	2.11	2.87
C7	4.07	6.97	3.08	3.85	3.48	2.78

*Only small endgames finished; these understate C0's cost.

TABLE IV
SELF-PLAY MATCHES (80 GAMES, 300 MS PER MOVE)

Match (first engine)	+	=	−	Score	Elo (95% CI)
Null move vs. none	49	11	20	68.1%	+132 (+61, +215)
4-ply vs. 3-ply guard	28	17	35	45.6%	−30 (−100, +37)
Final vs. earlier	42	17	21	63.1%	+93 (+27, +168)

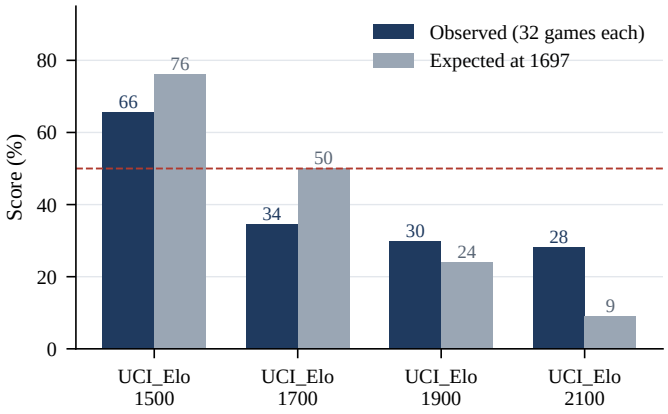


Fig. 7. Observed score against each Stockfish setting and the score expected at the pooled estimate of 1697. The misfit at the stronger settings is why the rating is reported as a range.

40% of the speed per node (365,774 nodes/s for C0, 147,974 for C4) because every node sorts its moves, yet it remains the best trade, cutting C1's depth-5 node count by a median of $5\times$.

D. Null-Move Pruning

The first implementation tried a null move whenever more than two plies remained, so the reduced search could fall directly into quiescence. On WAC #1 (1.Qg6! gxf6 2.Qh7#) that capture-only search could not see the quiet threat Qh7#, and the engine played 1.Ne8, needing depth 6 to find a mate it otherwise found at depth 4. Requiring four plies keeps every null search at least one full ply deep and restores the mate. Matches of 80 games at 300 ms per move over 40 openings with both colours (Table IV) show null move to be worth well over 100 Elo, with the stricter guard's cost within the noise.

E. Fixed-Size Transposition Table

Replacing an unbounded hash map with the fixed-size table exposed a defect: the Zobrist keys came from a linear congruential generator whose low bits are nearly periodic, and the fixed table selects slots by exactly those bits, so a 256-slot table filled to 50% and stayed there. The keys now come from the Mulberry32 generator, and a test requires every slot of a small table to be reachable. The shipped 2^{19} slots lie within 0.1–1.7% of the node counts of a table eight times larger.

F. Playing Strength

The reference was Stockfish 19 [5] with its calibrated `UCI_Elo` setting. Both engines had one second per move, and each of four anchors (1500–2100) played 32 games over 16 openings with both colours; ratings were estimated by maximum likelihood under the logistic model [19]. The pooled estimate is 1697 ± 72, while the two anchors that bracket the engine give 1600 ± 89. Fig. 7 shows why: the stronger settings lose far more often than their ratings predict, consistent with a strength limiter that occasionally chooses a worse move on purpose. The defensible statement is a range of about 1600–1700 at one second per move.

TABLE V
STRIDE THREAT ANALYSIS

Entry / asset	Threat (STRIDE)	Risk	Mitigation (status)
npm packages (E2)	Compromised package in build or bundle (T, E)	Medium	Lockfile install, tests before build (in place); scanning (recommended)
CI secret	Leaked deploy token (I)	Medium	Encrypted, masked secret (in place); scoped account (recommended)
Workflow (E1)	Unmerged branch deploys via pull-request trigger (T, E)	Medium	Deploy only on push to main (recommended) [21]
Fonts (E5)	Request reveals visitor IP; no CSP (I)	Low	Self-host fonts; add CSP [22] (recommended)
Transport (E4)	Modified or stale code (T)	Low	HTTPS; no-cache HTML, fingerprinted assets (in place)
Input, FEN (E6)	Injection or malformed FEN (T)	Low	React escaping; FEN validation (in place)
Worker	Expensive searches (D)	Low	Time budget, 60 s cap, fixed 10 MB table (in place)

VII. SECURITY ANALYSIS

Although the engine is an AI artefact, it is delivered as a public web application and is analysed with STRIDE [20]. It has no backend, accounts or stored personal data, so game state never leaves the browser. Fig. 8 shows three trust boundaries—GitHub, Firebase Hosting and the browser sandbox—crossed at six entry points, and Table V summarises the principal threats.

Three controls were added during this work: tests gate every deployment, FEN and typed moves are validated, and the table's memory is bounded. Two findings remain open—deployment on pull-request events and the absence of a Content-Security-Policy—and both can be closed with small configuration changes.

VIII. DISCUSSION AND LIMITATIONS

The experiments support the classical account of adversarial search: plain minimax pays the full branching factor, alpha-beta helps only once ordering lets it cut early, and the transposition table's value grows with depth. Null move is the one technique here that trades exactness for depth, and both sides

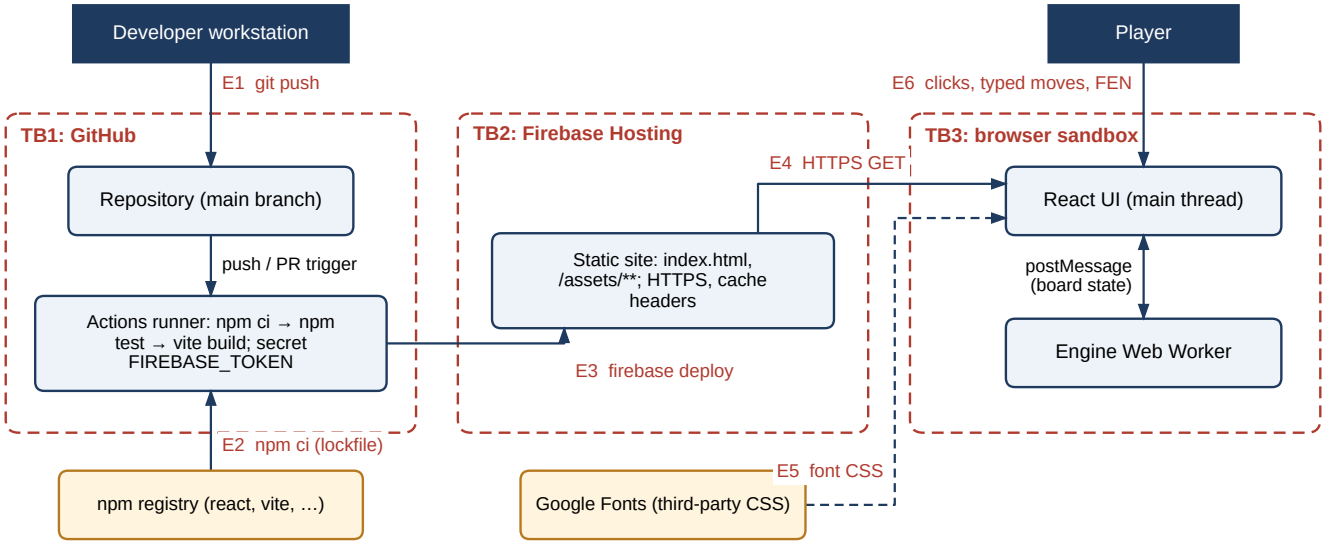


Fig. 8. Attack surface and trust boundaries of the deployed system, with entry points E1–E6.

of that trade were observed. Both defects reported here were found by measurement, not inspection, and are now pinned by tests. Limitations remain: quiescence ignores checks; quiet moves are searched without killer or history heuristics [10]; the Zobrist key is recomputed at every node; and the rating applies to one reference family, one time control and 128 games.

IX. Conclusion

This paper presented a transparent classical chess engine that is verified against published perft counts, guarded by 53 automated tests and measured technique by technique. Across 17 positions, alpha-beta with ordering and a transposition table matched minimax exactly while cutting the depth-4 tree $110.6\times$ and the per-ply cost from 24–27 to 2–5; null-move pruning added about 130 Elo; and the engine plays at roughly 1600–1700 on Stockfish's calibrated scale at one second per move. Future work will search check evasions in quiescence, add killer and history ordering, update hash keys incrementally and close the two open security findings.

Acknowledgment

The authors thank the Department of Computer Science and Engineering (Cyber Security), Rajalakshmi Engineering College, for its support, and the Stockfish developers, whose open-source engine served as the reference opponent.

References

[1] C. E. Shannon, "Programming a computer for playing chess," *Philosophical Magazine, Series 7*, vol. 41, no. 314, pp. 256–275, 1950.
[2] M. Campbell, A. J. Hoane Jr., and F. hsiung Hsu, "Deep Blue," *Artificial Intelligence*, vol. 134, no. 1–2, pp. 57–83, 2002.
[3] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
[4] Y. Nasu, "Efficiently updatable neural-network-based evaluation functions for computer shogi," The 28th World Computer Shogi Championship Appeal Document, 2018.
[5] The Stockfish developers, "Stockfish: Open-source chess engine," accessed: Sep. 2026. [Online]. Available: https://stockfishchess.org
[6] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.
[7] T. A. Marsland, "A review of game-tree pruning," *ICCA Journal*, vol. 9, no. 1, pp. 3–19, 1986.
[8] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.
[9] A. L. Zobrist, "A new hashing method with application for game playing," Computer Sciences Department, University of Wisconsin, Madison, WI, USA, Tech. Rep. 88, 1970.
[10] J. Schaeffer, "The history heuristic and alpha-beta search enhancements in practice," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 11, pp. 1203–1212, 1989.
[11] D. F. Beal, "A generalised quiescence search algorithm," *Artificial Intelligence*, vol. 43, no. 1, pp. 85–98, 1990.
[12] C. Donninger, "Null move and deep search: Selective-search heuristics for obtuse chess programs," *ICCA Journal*, vol. 16, no. 3, pp. 137–143, 1993.
[13] R. Coulom, "Efficient selectivity and backup operators in Monte-Carlo tree search," in *Computers and Games (CG 2006)*, ser. Lecture Notes in Computer Science, vol. 4630. Springer, 2007, pp. 72–83.
[14] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.
[15] T. Michniewski, "Simplified evaluation function," Chess Programming Wiki, accessed: Sep. 2026. [Online]. Available: https://www.chessprogramming.org/Simplified_Evaluation_Function
[16] Chess Programming Wiki, "Perft results," accessed: Sep. 2026. [Online]. Available: https://www.chessprogramming.org/Perft_Results
[17] F. Reinfeld, *Win at Chess*. New York, NY, USA: Dover, 1958.
[18] R. Fine, *Basic Chess Endings*. Philadelphia, PA, USA: David McKay, 1941.
[19] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York, NY, USA: Arco, 1978.
[20] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.
[21] GitHub, "Security hardening for GitHub Actions," accessed: Sep. 2026. [Online]. Available: https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions
[22] MDN Web Docs, "Content security policy (CSP)," accessed: Sep. 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP